Remindoo — Period Dates, Deadlines & Merge Tags

Development Specification v1.0

Owner: Waqas Sagar **Stack:** Node.js backend, existing recurring tasks module **Estimated effort:** 2 days backend, 1.5 days frontend

---

1. Problem

Recurring tasks currently store only **Period Ended (period_to)**. Email templates need Period From, soft/hard client deadlines, statutory filing deadlines, and internal deadlines — and firms want to set these once globally rather than per task.

2. Core Design Principle

Do not store Period From or deadlines on the recurring task template. Derive everything from `period_to` + frequency + offsets.

- Period boundaries are calendar-defined → derive from frequency (a quarter is not 90 days; offsets would be wrong 2 quarters out of 4).
- Deadlines are offset-defined → derive from statutory/firm offsets (+9 months, +1 month 7 days, –10 days etc.).
- One source of truth (`period_to`) means the recurrence roll-forward job needs **zero changes** — everything recomputes automatically each cycle.

3. Three-Level Settings Cascade

Resolution order: **Instance snapshot → Task override → Global firm default → System default.** First non-null wins.

Level 1 — Global firm settings

New JSON blob on firm settings, editable under **Settings → Deadlines**:

```
// firm_settings.deadline_defaults
{
```

```
  accounts:  { soft_months: 3, hard_months: 6, filing_months: 9,  internal_buffer
  ct600:     { filing_months: 12, internal_buffer_days: 10 },
  vat:       { filing: { months: 1, days: 7 }, internal_buffer_days: 5 },
  sa_return: { soft: 'fixed:30-06', hard: 'fixed:30-09', filing: 'fixed:31-01', i
  confirmation: { filing: { days: 14 } }
}
```

Ship these statutory values as system defaults. Firm edits once → applies to all tasks that haven't overridden.

Level 2 — Recurring task template

All offset fields **nullable**. `null` = inherit global. Only store a value when the user explicitly edits that task.

Level 3 — Generated instance (snapshot)

When the scheduler spawns each occurrence, resolve the cascade and **snapshot the computed dates onto the instance**. Rationale: if the firm changes global settings mid-year, future occurrences pick it up, but already-generated tasks (with emails possibly scheduled) stay stable. Add a "Recalculate dates" action on an instance for the rare manual refresh.

4. Database Changes

firm_settings

- `deadline_defaults` — JSON (structure above)

recurring_tasks (template) — all nullable

- `task_type` — enum: accounts | ct600 | vat | sa_return | confirmation | custom
- `period_from` — date (manual override only; normally null)
- `period_from_offset_days` — int (only for custom frequency)
- `soft_offset_months` — int
- `hard_offset_months` — int
- `filing_offset` — JSON `{ months, days }` or string `'fixed:DD-MM'`
- `internal_buffer_days` — int
- `due_offset_days` — int (existing +30 days due-date case)

task_instances (snapshot at generation)

- `period_from` — date
- `period_to` — date
- `soft_deadline` — date
- `hard_deadline` — date
- `filing_deadline` — date
- `internal_deadline` — date

5. Backend Logic (Node)

Use **date-fns** with `parseISO` on `YYYY-MM-DD` strings. **Date-only math throughout — never `new Date('2026-03-31')` with local-timezone arithmetic** (UTC server + BST display = off-by-one bugs).

5.1 Period From derivation

```
const { subMonths, subYears, subDays, addDays, addMonths, parseISO, format } = re

function getPeriodFrom(task) {
  if (task.period_from) return parseISO(task.period_from); // manual override

  const to = parseISO(task.period_to);

  switch (task.frequency) {
    case 'monthly':   return addDays(subMonths(to, 1), 1);
    case 'quarterly': return addDays(subMonths(to, 3), 1);
    case 'annually':  return addDays(subYears(to, 1), 1);
    default:
      return task.period_from_offset_days
        ? subDays(to, task.period_from_offset_days)
        : null;
  }
}
```

date-fns `subMonths` clamps month-ends correctly:

- 31 Mar − 3m + 1d = **1 Jan** ✓
- 31 May − 3m + 1d = **1 Mar** ✓ (Feb clamp)

- Year ended 31 Mar 2026 → from **1 Apr 2025** ✓

5.2 Cascade resolver

```
function resolveOffsets(task, firmSettings) {
  const global = firmSettings.deadline_defaults[task.task_type] ?? {};
  return {
    soft:   task.soft_offset_months  ?? global.soft_months  ?? null,
    hard:   task.hard_offset_months  ?? global.hard_months  ?? null,
    filing: task.filing_offset       ?? global.filing       ?? { months: global.f
    buffer: task.internal_buffer_days ?? global.internal_buffer_days ?? 10,
  };
}
```

5.3 Offset application

```
function applyOffset(baseDate, offset) {
  if (!offset) return null;
  if (typeof offset === 'string' && offset.startsWith('fixed:')) {
    // 'fixed:31-01' → next 31 Jan on/after baseDate
    const [dd, mm] = offset.slice(6).split('-').map(Number);
    let d = new Date(Date.UTC(baseDate.getUTCFullYear(), mm - 1, dd));
    if (d <= baseDate) d = new Date(Date.UTC(baseDate.getUTCFullYear() + 1, mm -
    return d;
  }
  if (typeof offset === 'number') return addMonths(baseDate, offset);
  let d = baseDate;
  if (offset.months) d = addMonths(d, offset.months);
  if (offset.days)   d = addDays(d, offset.days);
  return d;
}
```

5.4 Merge tag builder (single source of truth)

```
function buildMergeTags(task, firmSettings) {
  const to = parseISO(task.period_to);
  const from = getPeriodFrom(task);
  const o = resolveOffsets(task, firmSettings);

  const filing = applyOffset(to, o.filing);
  const fmt = d => d ? format(d, 'd MMMM yyyy') : '';

  return {
    period_from:       fmt(from),
```

```
    period_to:         fmt(to),
    soft_deadline:     fmt(applyOffset(to, o.soft)),
    hard_deadline:     fmt(applyOffset(to, o.hard)),
    filing_deadline:   fmt(filing),
    internal_deadline: fmt(filing ? subDays(filing, o.buffer) : null),
    tax_year:          getTaxYear(to),
    due_date:          fmt(task.due_offset_days ? addDays(to, task.due_offset_day
  };
}

function getTaxYear(periodTo) {
  // UK tax year: 6 Apr – 5 Apr
  const y = (periodTo.getUTCMonth() > 3 ||
             (periodTo.getUTCMonth() === 3 && periodTo.getUTCDate() >= 6))
    ? periodTo.getUTCFullYear()
    : periodTo.getUTCFullYear() - 1;
  return `${y}/${String(y + 1).slice(2)}`;
}
```

Worked example — accounts task, Period To = 31 Mar 2026, firm defaults:

Tag	Value
`{{period_from}}`	1 April 2025
`{{period_to}}`	31 March 2026
`{{soft_deadline}}` (+3m)	30 June 2026
`{{hard_deadline}}` (+6m)	30 September 2026
`{{filing_deadline}}` (+9m)	31 December 2026
`{{internal_deadline}}` (–10d)	21 December 2026
`{{tax_year}}`	2025/26

5.5 Recurrence job hook

In the existing roll-forward job, after advancing `period_to`:

```
const dates = buildMergeTags(task, firmSettings);
await TaskInstance.create({
  task_id: task.id,
  period_from: dates.period_from_raw,   // store raw dates, not formatted strings
  period_to:   task.period_to,
```

```
    soft_deadline: ..., hard_deadline: ...,
    filing_deadline: ..., internal_deadline: ...,
  });
```

(Adapt `buildMergeTags` to also return raw `Date` objects alongside formatted strings.)

5.6 Email/SMS rendering

At send time, replace `{{tag}}` placeholders from the **instance snapshot** (not recomputed). Reminder scheduling rules ("send 1 day after period end", "chase 14 days before internal deadline") also read the instance dates — so the ±N-day reminder timing rides on this for free.

5.7 API

- Expose computed dates in task/instance serializers.
- `GET /tasks/:id/computed-dates` — for frontend preview if the cascade logic isn't shipped to the client. Preferred: port `buildMergeTags` to a shared JS util so frontend preview exactly matches backend send.
- `POST /instances/:id/recalculate-dates` — manual refresh from current settings.

6. Frontend

6.1 Recurring task form — computed date chips

Below Period To + Frequency, a grey read-only summary strip that live-updates as inputs change:

```
Period: 1 Apr 2025 → 31 Mar 2026
Send info by: 30 Jun 2026 · Latest: 30 Sep 2026
Filing deadline: 31 Dec 2026 · Internal: 21 Dec 2026
```

- Each chip shows an inheritance badge: `(firm default)` greyed vs `(custom)` solid.
- Small ✏️ flips a chip to an editable offset input ("+3 months" / "−10 days") with a "reset to default" link that nulls the override.
- One component watching `period_to`, `frequency`, `task_type`.

6.2 Global settings screen — Settings → Deadlines

Table of task types × offsets (soft, hard, filing, internal buffer), pre-filled with statutory

defaults. Editing here updates `firm_settings.deadline_defaults` and flows to all non-overridden tasks and all future instances.

6.3 Template editor — merge tag picker

`{ } Insert tag` toolbar button → dropdown listing each tag with a live example value (computed from a sample task):

```
Period From        1 Apr 2025
Period To          31 Mar 2026
Soft deadline      30 Jun 2026
Hard deadline      30 Sep 2026
Filing deadline    31 Dec 2026
Internal deadline  21 Dec 2026
Tax year           2025/26
```

Click inserts `{{tag}}` at cursor.

6.4 Template preview tab

Renders the template with tags replaced using a real or sample client's instance dates. Highlight replaced values in light yellow so the user can verify before saving. Must call the same shared `buildMergeTags` util as the backend.

7. Behaviour Rules Summary

1. `period_from` is derived from `period_to` + frequency; never stored on the template (nullable override only).
2. Deadlines are offsets, resolved via cascade: instance → task → firm → system.
3. Instances snapshot dates at generation; templates stay live-derived.
4. Global settings changes affect future instances only (plus manual recalculate).
5. Roll-forward advances `period_to` only; everything else recomputes.
6. All date math is date-only / UTC-safe; format as `d MMMM yyyy` for client-facing output.

8. Test Cases

#	Scenario	Expected
1	Quarterly, to = 31 Mar 2026	from = 1 Jan 2026

2	Quarterly, to = 31 May 2026	from = 1 Mar 2026 (Feb clamp)
3	Annual, to = 31 Mar 2026	from = 1 Apr 2025
4	Accounts, to = 31 Mar 2026, defaults	filing = 31 Dec 2026, internal = 21 Dec 2026
5	VAT, to = 30 Jun 2026	filing = 7 Aug 2026 (1m 7d)
6	SA return, to = 5 Apr 2026	tax_year = 2025/26, filing = 31 Jan 2027, soft = 30 Jun 2026, hard = 30 Sep 2026
7	Task overrides soft to +2m	soft = 31 May 2026; badge shows (custom)
8	Firm changes internal buffer 10→14 after instance generated	existing instance unchanged; next occurrence uses 14
9	Recalculate button on instance from #8	instance updates to 14-day buffer
10	Server UTC, template rendered in BST	no off-by-one on any date

9. Delivery Plan

Phase	Work	Owner suggestion	Est.
1	Migrations + `buildMergeTags` + cascade resolver + unit tests (§8)	Sami / Qaim	1 day
2	Recurrence job hook + instance snapshot + render pipeline + APIs	Sami / Qaim	1 day
3	Date chips + Settings → Deadlines screen	Frontend	1 day
4	Merge tag picker + preview tab	Frontend	0.5 day